

Reading One Line: the Catalog

Every option at every one of the seven questions, each with a concrete example, real values, and what you end up holding

October 2026

How to use. At each question, find the row that matches the line in front of you. Every row has the same four parts: the number, the form, a **concrete example with real values**, and **what happens or what the label holds afterward**. Read the example, say the result out loud, and compare. Where a row shows two results (a safe case and a failing case), both are listed because the contrast is the lesson. Assume `users: User[]`, `orders: Order[]`, `req` is an Express request, and `getUser`, `save`, `load` are `async`.

Q1. What kind of line is it?

Declarations (a new label or type comes into existence)

#	Form	Concrete example	What it does / what is held
1	<code>const</code> with value	<code>const a = 1;</code>	<code>a</code> holds 1. A later <code>a = 2;</code> is a <code>TypeError</code> (label locked)
2	<code>let</code> with value	<code>let a = 1; a = 2;</code>	<code>a</code> holds 2 afterward; <code>let</code> allows re-pointing
3	<code>let</code> without value	<code>let a; then a</code>	<code>a</code> holds <code>undefined</code> until assigned. <code>let a: number;</code> then reading <code>a</code> before any assignment is a compile error
4	<code>var</code>	<code>if (true) { var a = 1; } then a</code>	<code>a</code> is 1 OUTSIDE the block (function-scoped). With <code>let</code> it would be a <code>ReferenceError</code> outside. Smell in modern code
5	destructuring object	<code>const { a, b } = { a: 1, b: 2 };</code>	<code>a</code> holds 1, <code>b</code> holds 2. Two labels created
6	destructuring with rename	<code>const { a: x } = { a: 7 };</code>	<code>x</code> holds 7. No label <code>a</code> exists; reading <code>a</code> is a <code>ReferenceError</code>
7	destructuring with default	<code>const { a = 5 } = {};</code>	<code>a</code> holds 5. With <code>{ a: 0 }</code> it holds 0; with <code>{ a: null }</code> it holds <code>null</code> (default only for <code>undefined</code>)
8	destructuring rest	<code>const { a, ...rest } = { a: 1, b: 2, c: 3 };</code>	<code>a</code> holds 1; <code>rest</code> holds a NEW object <code>{ b: 2, c: 3 }</code>
9	destructuring array	<code>const [x, y] = [10, 20, 30];</code>	<code>x</code> holds 10, <code>y</code> holds 20, the 30 is ignored
10	destructuring array rest	<code>const [h, ...t] = [1, 2, 3];</code>	<code>h</code> holds 1; <code>t</code> holds a NEW array <code>[2, 3]</code>
11	function declaration	<code>function f(x: number) { return x * 2 }</code>	<code>f(3)</code> is 6. Hoisted: <code>f(3)</code> works on a line ABOVE the declaration
12	async function declaration	<code>async function f() { return 5 }</code>	<code>f()</code> is a <code>Promise</code> (not 5). <code>await f()</code> is 5
13	generator	<code>function* g() { yield 1; yield 2; }</code>	<code>g()</code> is an iterator, not a number. <code>[...g()]</code> is <code>[1, 2]</code>
14	class	<code>class A { x = 1 }</code>	<code>new A().x</code> is 1. <code>A</code> is a constructor function; <code>A()</code> without <code>new</code> is a <code>TypeError</code>
15	class field	<code>class A { count = 0 }</code> and <code>const p = new A(); const q = new A(); p.count++;</code>	<code>p.count</code> is 1, <code>q.count</code> is still 0: each instance has its own field
16	class method	<code>class A { n = 5; m() { return this.n } }</code> and <code>const a = new A();</code>	<code>a.m()</code> is 5. <code>const f = a.m; f()</code> is a <code>TypeError</code> (<code>this</code> is <code>undefined</code> : method detached)
17	getter / setter	<code>class A { get total() { return 7 } }</code>	<code>new A().total</code> is 7. <code>new A().total()</code> is a <code>TypeError</code> (7 is not a function)
18	static member	<code>class A { static n = 0; static f() { return 1 } }</code>	<code>A.n</code> is 0, <code>A.f()</code> is 1. <code>new A().f</code> is <code>undefined</code> (not on instances)
19	<code>interface</code>	<code>interface U { id: number }</code> and <code>const u: U = { id: "x" };</code>	compile error on the second line. At runtime the interface does not exist and nothing is checked
20	<code>type</code> alias	<code>type Id = string number;</code> and <code>let i: Id = 5; i = true;</code>	first assignment fine; <code>i = true</code> is a compile error. Type vanishes at runtime
21	<code>enum</code>	<code>enum C { A, B }</code>	<code>C.A</code> is 0, <code>C.B</code> is 1, <code>C[0]</code> is "A". A real object at runtime
22	<code>import</code>	<code>import { x } from "./m";</code> and <code>import y from "./m";</code>	<code>x</code> is the module's named export <code>x</code> ; <code>y</code> is its default export. Same value, shared with every importer
23	<code>export</code>	<code>export const a = 1;</code> and <code>export default f;</code>	other files can <code>import { a }</code> and <code>import f</code> . Visibility only; nothing else changes

#	Form	Concrete example	What it does / what is held
24	declare	declare const g: string; and g.length	compiler trusts that g exists. If it does not exist at runtime, g.length is a ReferenceError

Destructuring, worked (every shape, with the input and what each label holds)

Statement	Input	Labels created	What each holds	What to check
const { id, name } = user;	user = { id: 1, name: "Ann", age: 30 }	id, name	id = 1, name = "Ann"; age is ignored	Q5: if name?: is optional, name may be undefined
const { id, name } = user;	user = { id: 1 }	id, name	id = 1, name = undefined	no error on a missing key; the crash comes at the next name.length
const { id } = user;	user = undefined	none	THROWS TypeError	Q5: can user be nothing? (find miss, optional arg)
const { a: x, b: y } = o;	o = { a: 1, b: 2 }	x, y	x = 1, y = 2; a, b do not exist as labels	reading a later is a ReferenceError
const { a = 5, b = 6 } = o;	o = { a: 1 }	a, b	a = 1, b = 6	default only fires on undefined
const { a = 5 } = o;	o = { a: null }	a	null, NOT 5	a null from JSON or a DB slips past the default
const { a: x = 5 } = o;	o = { }	x	5	rename and default together: x is the label
const { a, ...rest } = o;	o = { a: 1, b: 2, c: 3 }	a, rest	a = 1, rest = new { b: 2, c: 3 }	rest is shallow: nested objects shared with o
const { inner } = o;	o = { inner: { x: 1 } }	inner	the SAME object as o.inner	Q6: alias; inner.x = 2 changes o.inner.x
const { a: { b } } = o;	o = { a: { b: 5 } }	b (not a)	5	o = { } THROWS (reading b of undefined)
const { length } = "abc";	a string	length	3	works on primitives (temporary wrapper)
const [x, y] = arr;	arr = [10, 20, 30]	x, y	10, 20	30 ignored
const [x, y] = arr;	arr = [10]	x, y	10, undefined	Q5 on y
const [x, y] = arr;	arr = []	x, y	both undefined	empty array is not an error
const [x, , z] = arr;	arr = [1, 2, 3]	x, z	1, 3	the gap skips index 1
const [h, ...t] = arr;	arr = [1, 2, 3]	h, t	1, new [2, 3]	t is a copy; arr untouched
const [h, ...t] = arr;	arr = []	h, t	undefined , []	h can be nothing; t never is
const [first] = list.filter(p);	filter keeps 0 items	first	undefined	the idiom for "first match"; Q5
const [k, v] = pair;	pair = ["a", 1]	k, v	"a", 1	the Python k, v = pair
for (const [k, v] of Object.entries(o))	o = { a: 1, b: 2 }	k, v per pass	pass 1: "a", 1; pass 2: "b", 2	the Python d.items() loop
const [a, b] = [b, a];	after a = 1, b = 2 (needs let , and a prior ;)	rebinds a , b	a = 2, b = 1	swap without a temp; a missing semicolon on the previous line breaks it
function f({ id, tags = [] }: Opts) {}	f({ id: 1 })	id, tags inside f	1, a fresh []	calling f() with NO argument THROWS; add = {} after the pattern
function f([a, b]: [number, number]) {}	f([1, 2])	a, b	1, 2	wrong-length input gives undefined s, not errors
const { data: { items = [] } = {} } = res;	res = {}	items	[]	nested default on both levels prevents the crash
const { a } = o as Config;	o is really {}	a	undefined	the cast hid it; Q5

The three rules to carry

1. Missing key or missing element gives **undefined**, never an error.

2. Destructuring `undefined` or `null` itself THROWS; that is the crash to look for.
3. A destructured object-valued label is an ALIAS of the original's inner object; only the top level is new when you use `...rest`.

Assignments (an existing label re-pointed, or an object mutated)

#	Form	Concrete example	What it does / what is held
25	reassignment	<code>let a = 1; a = 2;</code>	<code>a</code> holds 2. Under <code>const</code> this line is a <code>TypeError</code>
26	compound	<code>let a = 5; a += 1; and let s = "x"; s += "y";</code>	<code>a</code> is 6; <code>s</code> is "xy". Each is a reassignment: <code>a = a + 1</code>
27	increment	<code>let i = 0; const j = i++;</code>	<code>j</code> is 0 (post-increment returns the OLD value), <code>i</code> is 1. <code>++i</code> would give <code>j</code> = 1
28	logical assignment	<code>let a = null; a ??= 5; and let b = 0; b = 5; and let c = 1; c &&= 9;</code>	<code>a</code> is 5 (was nullish); <code>b</code> is 5 (0 is falsy); <code>c</code> is 9 (1 is truthy). With <code>let d = 0; d ??= 5; d</code> stays 0
29	property set	<code>const o = { k: 1 }; const p = o; o.k = 2;</code>	<code>o.k</code> is 2 AND <code>p.k</code> is 2 (same object). Allowed under <code>const</code> : the label did not move
30	bracket set	<code>const o = {}; o["k"] = 1; and const a = [0, 0]; a[1] = "x";</code>	<code>o</code> is { k: 1 }; <code>a</code> is [0, "x"]
31	nested set	<code>const o = { a: { b: 1 } }; const inner = o.a; o.a.b = 2;</code>	<code>o.a.b</code> is 2 AND <code>inner.b</code> is 2: <code>inner</code> is another label on <code>o.a</code>
32	delete	<code>const o = { k: 1 }; delete o.k;</code>	<code>o</code> is {}; <code>o.k</code> is <code>undefined</code> . On a <code>Map</code> this does nothing; <code>map.delete(k)</code> is the call
33	length set	<code>const a = [1, 2, 3]; const b = a; a.length = 0;</code>	<code>a</code> is [] and so is <code>b</code> (same array). Empties every alias
34	destructuring assignment	<code>let a = 1, b = 2; [a, b] = [b, a];</code>	<code>a</code> is 2, <code>b</code> is 1. The right side builds [2, 1] first, then assigns
35	chained	<code>let a, b; a = b = 0;</code>	<code>b = 0</code> runs first and evaluates to 0; then <code>a = 0</code> . Both hold 0

Expression statements (a bare expression; something runs; the result is dropped unless the call mutates)

#	Form	Concrete example	What it does / what is held
36	mutating method call	<code>const a = [1]; a.push(2);</code>	<code>a</code> is now [1, 2]. The return value (2, the new length) is dropped; the mutation is the point
37	non-mutating method call	<code>const a = [1, 2]; a.map(x => x * 2);</code>	a new array [2, 4] is built and dropped. <code>a</code> is still [1, 2]. Dead line
38	sync side-effect call	<code>console.log(5); and res.json({ ok: true });</code>	first prints 5 and returns <code>undefined</code> ; second sends the HTTP response. The effect is the point
39	async call, no await	<code>save(x);</code>	starts <code>save</code> , gets a Promise, drops it. The next line runs before <code>save</code> finishes; if <code>save</code> rejects, unhandled rejection
40	<code>await</code> alone	<code>await save(x);</code>	waits for <code>save</code> ; the resolved value is dropped on purpose
41	<code>new</code> alone	<code>new Worker("w.js");</code>	starts the worker; the object is dropped (fine if the constructor's effect is the point)
42	bare value	<code>x; and a + b; and o.k;</code>	computed and discarded; the line does nothing
43	<code>void</code> expression	<code>void save(x);</code>	starts <code>save</code> , explicitly discards the Promise. If it rejects, still unhandled
44	IIFE	<code>(async () => { await save(x); })();</code>	runs immediately; the async function's Promise is dropped; errors inside are unhandled unless caught inside

Control (changes which line runs next)

#	Form	Concrete example	What it does / what is held
45	<code>if / else</code>	<code>const x = 0; if (x) { A() } else { B() }</code>	<code>B()</code> runs (0 is falsy). With <code>x = []</code> <code>A()</code> runs (arrays truthy). With <code>x = "0"</code> <code>A()</code> runs
46	ternary	<code>const s = n > 0 ? "pos" : "neg";</code>	<code>n = 5</code> gives "pos"; <code>n = 0</code> gives "neg"; it is an expression with a value
47	<code>switch</code>	<code>switch (x) { case 1: a(); case 2: b(); }</code>	<code>x = 1</code> runs <code>a()</code> THEN <code>b()</code> (no <code>break</code> : fall-through). <code>x = "1"</code> runs neither (===, string is not number 1)
48	<code>for</code>	<code>for (let i = 0; i <= arr.length; i++) { with arr = ["a", "b"]</code>	<code>i</code> takes 0, 1, 2; <code>arr[2]</code> is <code>undefined</code> : one past the end (off-by-one, <code><=</code> should be <code><</code>)
49	<code>for...of</code>	<code>for (const x of [10, 20]) { }</code>	<code>x</code> is 10, then 20

#	Form	Concrete example	What it does / what is held
50	<code>for...in</code>	<code>for (const k in { a: 1, b: 2 }) {}</code> and <code>for (const k in [10, 20]) {}</code>	first gives <code>"a"</code> , <code>"b"</code> ; second gives <code>"0"</code> , <code>"1"</code> (strings, not numbers, not the elements)
51	<code>while</code>	<code>let n = 3; while (n > 0) n--;</code>	stops with <code>n = 0</code> . <code>while (true) {}</code> with no <code>break</code> never ends
52	<code>break</code> / <code>continue</code>	<code>for (const x of [1, 2, 3]) { if (x === 2) break; log(x); }</code>	logs 1 only. Inside <code>[1,2,3].forEach(x => { break; })</code> it is a <code>SyntaxError</code>
53	<code>return</code>	<code>function f() { return; }</code> and <code>function g() { return 5; }</code>	<code>f()</code> is <code>undefined</code> ; <code>g()</code> is 5. Ends the function immediately
54	<code>throw</code>	<code>function f() { throw new Error("x"); }</code> and <code>try { f(); } catch (e) { e.message }</code>	the <code>catch</code> gets the <code>Error</code> ; <code>e.message</code> is <code>"x"</code> . With no enclosing <code>try</code> , the process crashes
55	<code>try</code> / <code>catch</code> / <code>finally</code>	<code>try { throw 1 } catch (e) { log(e) }</code> <code>finally { cleanup() }</code>	<code>e</code> is 1; <code>log(1)</code> runs; <code>cleanup()</code> runs regardless, even after <code>return</code> or <code>throw</code>
56	labeled statement	<code>outer: for (const i of [1,2]) { for (const j of [1,2]) { if (j === 2) break outer; }</code>	<code>break outer</code> leaves BOTH loops; plain <code>break</code> would leave only the inner
57	<code>yield</code>	<code>function* g() { const x = yield 1; }</code> and <code>const it = g();</code>	<code>it.next()</code> is <code>{ value: 1, done: false }</code> ; the generator pauses at <code>yield</code>

Type-only (runs nothing; note what the author claims)

#	Form	Concrete example	What it does / what is held
58	annotation	<code>let x: number = 5; x = "a";</code>	<code>x = "a"</code> is a compile error. At runtime, the <code>: number</code> does not exist
59	return type	<code>function f(): string { return 5; }</code>	compile error; also a path with no <code>return</code> is an error under strict settings
60	cast	<code>const n = "5" as unknown as number; n + 1</code>	compiles; at runtime <code>n</code> is still the string <code>"5"</code> , and <code>n + 1</code> is <code>"51"</code> . The cast changed nothing
61	non-null	<code>const len = s!.length;</code> with <code>s</code> really <code>undefined</code>	compiles; at runtime <code>TypeError</code> . The <code>!</code> only silenced the compiler
62	<code>satisfies</code>	<code>const c = { port: 80 } satisfies Config;</code>	checks the shape against <code>Config</code> without widening: <code>c.port</code> stays typed <code>number</code> . An extra key is a compile error
63	generic parameter	<code>function id<T>(x: T): T { return x }</code>	<code>id(5)</code> returns 5 typed <code>number</code> ; <code>id("a")</code> returns <code>"a"</code> typed <code>string</code> . <code>T</code> is a placeholder
64	<code>as const</code>	<code>const r = ["a", "b"] as const;</code>	type becomes <code>readonly ["a", "b"]</code> ; at runtime it is an ordinary mutable array (<code>r.push("c")</code> works if cast away)
65	<code>typeof</code> in a type	<code>const c = { a: 1 }; let d: typeof c;</code>	<code>d</code> must have the type <code>{ a: number }</code> ; nothing runs
66	<code>keyof</code>	<code>type K = keyof { a: 1; b: 2 };</code>	<code>K</code> is <code>"a" "b"</code>
67	<code>@ts-ignore</code>	<code>// @ts-ignore above x.foo();</code>	the compiler skips the next line; at runtime <code>x.foo()</code> may crash. A finding

Q2. Where does the value come from?

Literals

#	Form	Concrete example	What it produces
1	string	<code>"a"</code> , <code>'a'</code> , <code>`a\${1}`</code>	<code>"a"</code> , <code>"a"</code> , <code>"a1"</code> (all primitives)
2	number	<code>42</code> , <code>3.5</code> , <code>-0</code> , <code>1e3</code> , <code>0x1f</code> , <code>NaN</code> , <code>Infinity</code>	<code>42</code> , <code>3.5</code> , <code>-0</code> , <code>1000</code> , <code>31</code> , <code>NaN</code> , <code>Infinity</code>
3	boolean	<code>true</code> , <code>false</code>	the two booleans
4	nullish	<code>null</code> , <code>undefined</code>	the two "nothing" values
5	bigint	<code>10n</code> and <code>10n + 1</code>	<code>10n</code> is a bigint; <code>10n + 1</code> is a <code>TypeError</code> (cannot mix with number)
6	object literal	<code>{}</code> , <code>{ a: 1 }</code> , <code>const a = 3; ({ a })</code> , <code>const k = "x"; ({ [k]: 1 })</code> , <code>{ ...o }</code>	<code>{}</code> , <code>{ a: 1 }</code> , <code>{ a: 3 }</code> , <code>{ x: 1 }</code> , a shallow copy of <code>o</code> . Each evaluation makes a NEW object
7	array literal	<code>[]</code> , <code>[1, 2]</code> , <code>[...[1], ...[2]]</code>	<code>[]</code> , <code>[1, 2]</code> , <code>[1, 2]</code> . Each evaluation makes a NEW array

#	Form	Concrete example	What it produces
8	function literal	<code>const f = (x) => x + 1;</code>	<code>f(1)</code> is 2. The arrow is a function object
9	regex literal	<code>"abb".match(/ab+/g)</code>	<code>["abb"]</code> . With no match: <code>null</code>
10	template literal	<code>`Hello \${name}`</code> with <code>name = "Ann"</code>	<code>"Hello Ann"</code> . With <code>name = undefined</code> : <code>"Hello undefined"</code> (no error)
11	new expression	<code>new Map(), new Set([1, 1]), new Date(0), new Error("m"), new User()</code>	an empty Map; <code>Set { 1 }</code> ; the epoch Date; an Error with message <code>"m"</code> ; a <code>User</code> instance. Each a NEW object

Calls

#	Form	Concrete example	What it produces
12	plain function call	<code>Math.max(1, 3)</code> and <code>f(2)</code> for <code>function f(x) { return x * 2 }</code>	3; 4
13	method call	<code>[3, 1].sort()</code> and <code>" a ".trim()</code>	<code>[1, 3]</code> (mutated the array and returned it); <code>"a"</code> (new string)
14	optional call	<code>const cb = undefined; cb?.(5)</code>	<code>undefined</code> , no crash. Without <code>?.</code> : <code>TypeError</code>
15	constructor	<code>new Date(0).getTime()</code>	0
16	chained calls	<code>[1, 2, 3].filter(x => x > 1).map(x => x * 2)</code>	<code>filter</code> gives <code>[2, 3]</code> , then <code>map</code> gives <code>[4, 6]</code> . The last call's return is the value
17	IIFE	<code>((() => 5)())</code>	5
18	tagged template	<code>sql`select \${id}`</code> with <code>id = 7</code>	calls <code>sql(["select ", "", 7])</code> ; the result is whatever <code>sql</code> returns
19	<code>await</code> of a call	<code>await getUser("1")</code>	the resolved <code>User</code> ; a rejection throws here
20	<code>.then</code> chain	<code>getUser("1").then(u => u.name)</code>	a Promise of the name, not the name
21	<code>Promise.all</code>	<code>Promise.all([getUser("1"), getUser("2")])</code>	a Promise of <code>[user1, user2]</code>
22	<code>JSON.parse</code>	<code>JSON.parse('{ "a":1 }')</code> and <code>JSON.parse("x")</code>	<code>{ a: 1 }</code> ; throws <code>SyntaxError</code> . The result is typed <code>any</code>
23	<code>Number(x)</code>	<code>Number("12")</code> , <code>Number("12px")</code> , <code>Number("")</code> , <code>Number(null)</code> , <code>Number(undefined)</code>	12, NaN, 0, 0, NaN
24	<code>parseInt</code> / <code>parseFloat</code>	<code>parseInt("12px", 10)</code> , <code>parseInt("px")</code> , <code>parseFloat("1.5.3")</code>	12, NaN, 1.5
25	<code>Object.keys/values/entries</code>	<code>Object.keys({ a: 1, b: 2 })</code> , <code>Object.values(...)</code> , <code>Object.entries(...)</code>	<code>["a", "b"]</code> , <code>[1, 2]</code> , <code>[["a", 1], ["b", 2]]</code> (new arrays)
26	<code>Array.from</code> / <code>isArray</code>	<code>Array.from("ab")</code> and <code>Array.isArray([])</code>	<code>["a", "b"]</code> ; <code>true</code>
27	<code>structuredClone</code>	<code>const o = { a: { b: 1 } }; const c = structuredClone(o);</code>	<code>c.a !== o.a</code> ; editing <code>c.a.b</code> leaves <code>o.a.b</code> alone (deep copy)
28	<code>Date.now()</code>	<code>Date.now()</code>	a number of milliseconds, e.g. 1760000000000
29	<code>Math.*</code>	<code>Math.max(...[])</code> , <code>Math.round(2.5)</code> , <code>Math.floor(-1.5)</code>	<code>-Infinity</code> , 3, -2

Property and element reads

#	Form	Concrete example	What it produces
30	dot	<code>const o = { k: 1 }; o.k</code> and <code>({}).k</code>	1; <code>undefined</code> (no error)
31	bracket, string key	<code>const name = "k"; o[name]</code> and <code>({}["constructor"])</code>	1; the inherited <code>Object</code> function (not <code>undefined</code>)
32	index	<code>[10, 20][0]</code> , <code>[10, 20][2]</code> , <code>[10, 20][-1]</code>	10, <code>undefined</code> , <code>undefined</code>
33	<code>arr.at(i)</code>	<code>[10, 20].at(-1)</code> and <code>[] .at(0)</code>	20; <code>undefined</code>
34	<code>.length</code>	<code>"abc".length</code> and <code>[1, 2].length</code>	3; 2 (always a number)
35	<code>.size</code>	<code>new Map([[1, 2]]).size</code> and <code>new Set().size</code>	1; 0

#	Form	Concrete example	What it produces
36	<code>map.get(k)</code>	<code>const m = new Map([["a", 1]]); m.get("a"); m.get("b")</code>	1; undefined
37	optional chain	<code>const o = undefined; o?.k, o?.a?.b, const arr = null; arr?.[0]</code>	all undefined, no crash
38	nested	<code>const o = { a: {} }; o.a.b.c</code>	<code>o.a.b</code> is undefined, then <code>.c</code> on it throws <code>TypeError</code>
39	request fields	URL <code>/users/42?page=2</code> : <code>req.params.id</code> , <code>req.query.page</code> , <code>req.query.zzz</code>	"42" (a string), "2" (a string), undefined. <code>req.headers["x-k"]</code> is a string, a string array, or undefined
40	<code>this.field</code>	<code>class A { items = [1]; get() { return this.items } }</code>	<code>new A().get()</code> is [1]. <code>const g = new A().get</code> ; <code>g()</code> is a <code>TypeError</code> (<code>this</code> lost)
41	<code>process.env.X</code>	<code>process.env.PORT</code>	"3000" (a string) or undefined if unset
42	<code>localStorage.getItem</code>	<code>localStorage.getItem("k")</code>	the stored string, or null if absent

Operators

#	Form	Concrete example	What it produces
43	arithmetic	<code>5 + 2</code> , <code>"5" + 2</code> , <code>"5" - 2</code> , <code>1 / 0</code> , <code>0 / 0</code> , <code>7 % 3</code> , <code>2 ** 3</code>	7, "52", 3, Infinity, NaN, 1, 8
44	string concatenation	<code>"a" + 1 + 2</code> and <code>1 + 2 + "a"</code>	"a12"; "3a" (left to right)
45	comparison	<code>3 > 2</code> , <code>NaN > 1</code> , <code>"10" < "9"</code> , <code>"10" < 9</code>	true, false, true (string compare), false (numeric compare)
46	strict equality	<code>1 === 1</code> , <code>1 === "1"</code> , <code>NaN === NaN</code>	true, false, false
47	loose equality	<code>0 == ""</code> , <code>null == undefined</code> , <code>[] == false</code>	true, true, true (coerced; flag in review)
48	<code>&&</code>	<code>0 && "x"</code> , <code>5 && "x"</code> , <code>user && user.name</code>	0 (stops at the first falsy), "x", the name if <code>user</code> is set else <code>user</code> itself
49	<code> </code>	<code>0 10</code> , <code>"" "anon"</code> , <code>5 10</code>	10, "anon", 5
50	<code>??</code>	<code>0 ?? 10</code> , <code>null ?? 10</code> , <code>undefined ?? 10</code> , <code>"" ?? "anon"</code>	0, 10, 10, ""
51	NOT	<code>!0</code> , <code>!"a"</code> , <code>!![]</code> , <code>!!""</code>	true, false, true, false
52	ternary	<code>5 > 3 ? "y" : "n"</code>	"y"
53	spread in a literal	<code>[...[1, 2], 3]</code> and <code>({ ...{ a: 1 }, b: 2 })</code>	[1, 2, 3]; { a: 1, b: 2 } (new containers, shallow)
54	<code>typeof</code>	<code>typeof 5</code> , <code>typeof "a"</code> , <code>typeof null</code> , <code>typeof undefined</code> , <code>typeof []</code> , <code>typeof (() => 1)</code>	"number", "string", "object" (wart), "undefined", "object", "function"
55	<code>instanceof</code>	<code>[] instanceof Array</code> , <code>new Date() instanceof Date</code> , <code>({}) instanceof Array</code>	true, true, false
56	<code>in</code>	<code>"a" in { a: 1 }</code> , <code>"toString" in {}</code> , <code>1 in [5, 6]</code>	true, true (inherited), true (an index)
57	<code>delete</code>	<code>const o = { k: 1 }; delete o.k</code>	true; o is {}
58	<code>void</code>	<code>void 0</code>	undefined
59	comma	<code>(1, 2)</code>	2
60	assignment as an expression	<code>let a; (a = 5)</code> and <code>let b = 9; if (b = 0) {}</code>	evaluates to 5 (and a is 5); the if is false AND b is now 0
61	bitwise	<code>5 & 3</code> , <code>5 3</code> , <code>1 << 3</code> , <code>~0</code>	1, 7, 8, -1

await and destructuring

#	Form	Concrete example	What it produces
62	<code>await p</code>	<code>await Promise.resolve(5)</code> and <code>await Promise.reject(new Error("x"))</code>	5; throws the Error at this line
63	<code>await</code> of a non-Promise	<code>await 5</code>	5, after yielding once to other work
64	<code>const { a } = o</code>	<code>const { a } = { a: 1 }</code>	a holds 1
65	<code>const [x] = arr</code>	<code>const [x] = [9]</code>	x holds 9

#	Form	Concrete example	What it produces
66	nested pattern	<code>const { a: { b } } = { a: { b: 1 } }</code> and <code>= {}</code>	<code>b</code> holds 1; the second throws <code>TypeError</code> (reading <code>b</code> of <code>undefined</code>)
67	<code>for (const [k, v] of map)</code>	<code>for (const [k, v] of new Map([["a", 1]]))</code>	<code>k</code> is <code>"a"</code> , <code>v</code> is 1
68	<code>for (const [k, v] of Object.entries(o))</code>	<code>Object.entries({ a: 1 })</code>	<code>k</code> is <code>"a"</code> , <code>v</code> is 1

Q3. Primitive or object?

Primitives (immutable, by value, copied, no keys)

#	Type	Concrete examples that produce it	Why that is this type
1	string	<code>"a".toUpperCase()</code> gives <code>"A"</code> ; <code>String(5)</code> gives <code>"5"</code> ; <code>[1, 2].join("-")</code> gives <code>"1-2"</code> ; <code>typeof 5</code> gives <code>"number"</code> ; <code>JSON.stringify({ a: 1 })</code> gives <code>'{"a":1}'</code>	every string method, <code>typeof</code> , and <code>JSON.stringify</code> return a string
2	number	<code>"abc".length</code> is 3; <code>[1].push(2)</code> is 2 (the new length); <code>"a".indexOf("b")</code> is -1; <code>Math.max(1, 2)</code> is 2; <code>Date.now()</code>	lengths, indexes, counts, math, and timestamps are numbers
3	boolean	<code>[1].includes(1)</code> is true; <code>new Map().has("a")</code> is false; <code>Array.isArray([])</code> ; <code>1 < 2</code> ; <code>delete o.k</code>	predicates and comparisons
4	undefined	<code>[]</code> .find(<code>x => x</code>); <code>new Map().get("a")</code> ; <code>[1][5]</code> ; <code>((() => {}))()</code> ; <code>({}).k</code> ; <code>null?.k</code>	a miss, an absent key, or no <code>return</code>
5	null	<code>JSON.parse("null")</code> ; <code>localStorage.getItem("none")</code> ; <code>"a".match(/b/)</code> ; <code>await db.findOne({ id: "none" })</code> ; <code>let x = null</code>	deliberately empty, or "not found" from an API
6	bigint	<code>10n</code> ; <code>BigInt(5)</code>	only from the <code>n</code> literal or <code>BigInt()</code>
7	symbol	<code>Symbol("id")</code> ; <code>Symbol.iterator</code>	unique keys; rare

Objects (mutable, by identity, aliased, can have keys)

#	Kind	Concrete examples that produce it	Why that is an object
8	plain object	<code>{ a: 1 }</code> ; <code>{ ...o }</code> ; <code>JSON.parse('{ "a":1 }')</code> ; <code>Object.fromEntries([["a", 1]])</code> ; <code>req.body</code>	a bag of keys
9	array	<code>[1, 2].map(x => x)</code> ; <code>"a,b".split(",")</code> gives <code>["a", "b"]</code> ; <code>Object.keys({ a: 1 })</code> ; <code>Array.from("ab")</code> ; <code>[3,1].toSorted()</code>	an ordered list
10	Map / Set	<code>new Map()</code> ; <code>new Set([1, 1])</code> ; <code>Map.groupBy(arr, f)</code>	collections
11	function	<code>() => {}</code> ; <code>obj.method</code> (named, not called); <code>f.bind(x)</code>	functions are objects; <code>typeof</code> says <code>"function"</code>
12	Promise	<code>fetch(url)</code> ; <code>res.json()</code> ; <code>(async () => 1)()</code> ; <code>Promise.all([])</code> ; <code>f().then(g)</code>	a ticket object
13	Date	<code>new Date(0)</code>	mutable: <code>d.setFullYear(2000)</code> changes it
14	Error	<code>new Error("m")</code> ; the <code>e</code> in <code>catch (e)</code> (whatever was thrown)	has <code>.message</code> , <code>.stack</code>
15	RegExp	<code>/x/g</code> ; <code>new RegExp("x")</code>	with <code>g</code> , carries <code>lastIndex</code> state
16	class instance	<code>new User()</code>	an object whose prototype is <code>User.prototype</code>
17	iterator	<code>[1].entries()</code> ; <code>new Map().keys()</code> ; <code>g()</code> for a generator	has <code>.next()</code>
18	web objects	<code>new URL("http://x")</code> ; <code>new Headers()</code> ; the Response from <code>await fetch(url)</code>	built-in classes
19	binary	<code>Buffer.from("a")</code> ; <code>new Uint8Array(2)</code>	byte arrays
20	framework objects	<code>req</code> , <code>res</code> , <code>new EventEmitter()</code> , a DB pool	provided by libraries

Q4. Ticket or value?

Returns a Promise (a ticket)

#	Form	Concrete example	What you hold
1	declared <code>async</code>	<code>async function f() { return 1 } then const v = f();</code>	<code>v</code> is a Promise, not 1. <code>await v</code> is 1
2	annotated <code>:</code> <code>Promise<T></code>	<code>function f(): Promise<number> { return Promise.resolve(1) } then const v = f();</code>	a Promise
3	<code>fetch</code>	<code>const r = fetch("/users/1");</code>	a Promise of a Response. After <code>await</code> , a Response with <code>status</code> 404 is still a Response
4	<code>res.json()</code>	<code>const d = res.json(); then d.user</code>	<code>d</code> is a Promise; <code>d.user</code> is <code>undefined</code>
5	<code>new Promise</code>	<code>const p = new Promise((resolve) => resolve(5));</code>	<code>p</code> is a Promise; <code>await p</code> is 5
6	<code>Promise.all</code> and friends	<code>const p = Promise.all([f(), g()]);</code>	one Promise of <code>[fValue, gValue]</code>
7	<code>.then / .catch / .finally</code>	<code>const p = f().then(x => x * 2);</code>	a Promise of the doubled value (chaining always yields a Promise)
8	<code>import()</code>	<code>const m = import("./m.js");</code>	a Promise of the module
9	<code>fs/promises</code>	<code>const text = readFile("a.txt", "utf8");</code>	a Promise of a string (<code>text.length</code> is <code>undefined</code>)
10	<code>util.promisify</code>	<code>const read = promisify(fs.readFile); const t = read("a.txt");</code>	a Promise
11	database client	<code>const rows = db.query("select 1");</code>	a Promise of rows; <code>rows.length</code> is <code>undefined</code>
12	HTTP client or SDK	<code>const r = axios.get(url); and const out = client.send(cmd);</code>	Promises
13	pass-through <code>return</code>	<code>function f() { return fetch(u); } then const v = f();</code>	<code>v</code> is a Promise although <code>f</code> has no <code>async</code>
14	<code>map</code> with async callback	<code>const ps = ["1", "2"].map(async id => getUser(id));</code>	<code>ps</code> is an ARRAY of two Promises; <code>ps[0].name</code> is <code>undefined</code>
15	async IIFE	<code>const p = (async () => { await save(); })();</code>	a Promise (dropped if the call is a bare statement)

Returns a value (not a ticket)

#	Form	Concrete example	What you hold
16	<code>JSON.parse</code>	<code>const o = JSON.parse('{ "a": 1 }');</code>	the object now; <code>o.a</code> is 1
17	array methods	<code>const b = [1, 2].map(x => x * 2);</code>	<code>[2, 4]</code> now. If the callback were async, <code>b</code> would hold Promises, but <code>map</code> itself is synchronous
18	string methods	<code>const u = "a".toUpperCase();</code>	<code>"A"</code> now
19	<code>Math.*</code> , <code>Date.now()</code>	<code>const m = Math.max(1, 2);</code>	2 now
20	<code>readFileSync</code>	<code>const t = fs.readFileSync("a.txt", "utf8");</code>	the string now; blocks the process while reading
21	timers	<code>const id = setTimeout(fn, 10);</code>	a timer id (number in browsers, a <code>Timeout</code> object in Node), not a Promise
22	Express <code>res.*</code>	<code>res.status(404).json({ error: "x" });</code>	sends the response; each call returns <code>res</code> ; none is a Promise; the function keeps running
23	misc sync	<code>localStorage.setItem("k", "v")</code> gives <code>undefined</code> ; <code>console.log(1)</code> gives <code>undefined</code> ; <code>crypto.randomUUID()</code> gives a string	values now
24	<code>new</code> of an ordinary class	<code>const m = new Map();</code>	the Map now

#	Form	Concrete example	What you hold
25	<code>forEach</code> with async callback	<code>const r = ids.forEach(async id => await save(id));</code>	<code>r</code> is <code>undefined</code> ; the Promises were dropped

How the ticket is handled on this line

#	Form	Concrete example	Verdict
26	<code>await</code> into a label	<code>const v = await f();</code> with <code>f</code> resolving to 1	<code>v</code> holds 1 (the value)
27	<code>return f()</code> inside <code>async</code>	<code>async function g() { try { return f(); } catch (e) { log(e) } }</code>	<code>f</code> 's Promise leaves <code>g</code> 's <code>try</code> pending; if <code>f</code> rejects, <code>log</code> never runs
28	<code>return await f()</code> inside <code>async</code>	<code>async function g() { try { return await f(); } catch (e) { log(e) } }</code>	if <code>f</code> rejects, the error is thrown inside the <code>try</code> ; <code>log</code> runs
29	<code>.then</code> chain with handlers	<code>f().then(g).catch(h);</code>	handled by the chain; the chain's own result is still a Promise
30	<code>await Promise.all</code>	<code>const all = await Promise.all([f(), g()]);</code>	<code>all</code> is <code>[1, 2]</code> ; parallel; rejects on the first failure
31	deferred <code>await</code>	<code>const p = f(); other(); const v = await p;</code>	<code>f</code> and <code>other</code> overlap in time; <code>v</code> is the value
32	bare <code>async</code> call	<code>f();</code>	dropped; if <code>f</code> rejects, unhandled rejection
33	ticket used as value	<code>const v = f(); v.name</code>	<code>undefined</code> (a Promise has no <code>name</code>)
34	<code>await</code> inside <code>forEach</code>	<code>ids.forEach(async id => { await save(id); }); log("done");</code>	"done" logs BEFORE any save finishes
35	<code>await</code> in a non- <code>async</code> function	<code>function g() { const v = await f(); }</code>	SyntaxError: <code>await</code> is only valid in an <code>async</code> function

Q5. Can it be nothing?

Yes, it can be `undefined`

#	Source	Concrete example	Result
1	optional property	<code>const u: { email?: string } = {}; u.email</code> and <code>u.email.length</code>	<code>undefined</code> ; then <code>TypeError</code> on <code>.length</code>
2	loose type	<code>const d = JSON.parse("{}"); d.user</code> and <code>d.user.name</code>	<code>undefined</code> ; then <code>TypeError</code>
3	<code>find</code> miss	<code>[1, 2].find(x => x > 5)</code> and <code> [].findLast(x => x)</code>	<code>undefined</code> , <code>undefined</code>
4	index out of range	<code>[1, 2][2]</code> and <code>[1, 2].at(5)</code>	<code>undefined</code> , <code>undefined</code>
5	always-undefined index	<code>[1, 2][-1]</code> and <code>arr[arr.length]</code>	<code>undefined</code> even though the array is non-empty
6	<code>Map.get</code> miss	<code>new Map().get("a")</code>	<code>undefined</code>
7	<code>pop</code> / <code>shift</code> on empty	<code> [].pop()</code> and <code> [].shift()</code>	<code>undefined</code> , <code>undefined</code>
8	request fields	<code>GET /users</code> with no query: <code>req.query.page</code> ; no header: <code>req.headers["x-key"]</code>	<code>undefined</code> , <code>undefined</code>
9	environment	<code>process.env.NOPE</code>	<code>undefined</code>
10	function falls off the end	<code>function f(x) { if (x) return 1; } then f(0)</code>	<code>undefined</code>
11	braces without <code>return</code>	<code>[1, 2].map(x => { x * 2 })</code>	<code>[undefined, undefined]</code>
12	declared, never assigned	<code>let x; x</code>	<code>undefined</code>
13	optional chain	<code>const o = null; o?.k</code>	<code>undefined</code>
14	destructuring a missing key	<code>const { a } = {};</code>	<code>a</code> holds <code>undefined</code>
15	default parameter	<code>function f(n = 5) { return n }; f(), f(undefined), f(null)</code>	5, 5, <code>null</code> (default fires only for <code>undefined</code>)
16	first of an empty list	<code>Object.values({})[0]</code>	<code>undefined</code>

#	Source	Concrete example	Result
17	missing regex group	<code>"ab".match(/(x)/)?.[1]</code>	<code>undefined</code> (match is <code>null</code> , <code>?.</code> stops)
18	never settles	<code>await Promise.race([])</code>	not <code>undefined</code> : the function hangs forever

Yes, it can be `null`

#	Source	Concrete example	Result
19	explicit <code>null</code>	<code>let x = null;</code>	<code>null</code>
20	database lookup	<code>await db.user.findOne({ id: "none" })</code>	<code>null</code>
21	<code>JSON.parse("null")</code>	<code>JSON.parse("null")</code>	<code>null</code>
22	<code>localStorage</code> miss	<code>localStorage.getItem("none")</code>	<code>null</code>
23	regex miss	<code>"abc".match(/x/) and /x/.exec("abc")</code>	<code>null, null</code>
24	DOM lookup	<code>document.querySelector("#none")</code>	<code>null</code>
25	declared <code>T null</code>	<code>function f(): string null { return Math.random() > 2 ? "a" : null }</code>	<code>null</code> (always, for this input)
26	<code>null</code> passed to a default	<code>function f(n = 5) { return n } f(null)</code>	<code>null</code> , NOT 5

No, it cannot be nothing (unless a type lies)

#	Source	Concrete example	Result
27	a literal	<code>const a = 5; a</code>	5
28	lengths and sizes	<code>{}.length</code> and <code>new Map().size</code>	0, 0 (numbers, never <code>undefined</code>)
29	array-returning methods	<code>{}.filter(x => x), Object.keys({}), [1].map(x => x)</code>	<code>[], [], [1]</code> (empty, but an array)
30	<code>new</code>	<code>new Date()</code>	a Date object
31	<code>Number(x)</code>	<code>Number("abc")</code>	<code>NaN</code> (a number; nothing-like, but not <code>undefined</code>)
32	<code>String(x)</code> and templates	<code>String(undefined)</code>	<code>"undefined"</code> (a string)
33	predicates	<code>[1].includes(2)</code>	<code>false</code>
34	awaited, honestly typed	<code>const u = await getUser("1")</code> with <code>getUser(): Promise<User></code> that never resolves empty	a <code>User</code>

How the next use handles nothing (test each with `x = undefined`, `x = null`, `x = 0`)

#	Form	<code>x = undefined</code>	<code>x = null</code>	<code>x = 0</code>	Verdict
35	<code>if (x) { ... }</code>	skipped	skipped	skipped	also skips <code>0</code> ; wrong when 0 is valid
36	<code>if (!x) return;</code>	returns	returns	returns	same: also catches <code>0</code> , <code>""</code> , <code>false</code>
37	<code>if (x === undefined)</code>	true	false	false	precise for <code>undefined</code> only
38	<code>if (x == null)</code>	true	true	false	the idiomatic "nothing" check
39	<code>x ?? "d"</code>	<code>"d"</code>	<code>"d"</code>	0	falls back only for nothing
40	<code>x "d"</code>	<code>"d"</code>	<code>"d"</code>	<code>"d"</code>	falls back for 0 too: wrong for counts
41	<code>x?.k</code>	<code>undefined</code>	<code>undefined</code>	<code>undefined</code> (0 has no <code>k</code>)	safe access; result must be handled
42	<code>x!.k</code>	<code>TypeError</code>	<code>TypeError</code>	<code>undefined</code>	the <code>!</code> checks nothing
43	<code>x.k</code>	<code>TypeError</code>	<code>TypeError</code>	<code>undefined</code>	crashes on nothing
44	<code>typeof x === "string"</code>	false	false	false	no crash; says "not a string"
45	<code>Array.isArray(x)</code>	false	false	false	no crash
46	<code>x instanceof Date</code>	false	false	false	no crash
47	<code>"k" in x</code>	<code>TypeError</code>	<code>TypeError</code>	<code>TypeError</code>	<code>in</code> needs an object on the right

Q6. Who else points here?

This line made a NEW address (own)

#	Form	Concrete example	Result
1	object literal	<code>const p = {}; const q = {};</code>	<code>p === q</code> is false : two objects
2	array literal	<code>const p = [1]; const q = [1];</code>	<code>p === q</code> is false
3	<code>{ ...o }</code>	<code>const o = { a: 1, i: { x: 1 } }; const c = { ...o };</code>	<code>c !== o</code> ; but <code>c.i === o.i</code> (inner object shared)
4	<code>[...a]</code>	<code>const a = [{ n: 1 }]; const c = [...a];</code>	<code>c !== a</code> ; but <code>c[0] === a[0]</code>
5	<code>slice</code> / <code>Array.from</code> / <code>concat</code>	<code>const a = [1, 2]; const c = a.slice();</code>	<code>c !== a</code> ; <code>c.push(3)</code> leaves <code>a</code> as <code>[1, 2]</code>
6	<code>map</code> / <code>filter</code>	<code>const users2 = users.filter(u => u.active);</code>	<code>users2 !== users</code> ; <code>users2[0] === users[i]</code> (same user objects)
7	<code>toSorted</code> / <code>toReversed</code>	<code>const a = [3, 1]; const s = a.toSorted();</code>	<code>s</code> is <code>[1, 3]</code> ; <code>a</code> is still <code>[3, 1]</code>
8	<code>Object.assign({}, o)</code> and friends	<code>const c = Object.assign({}, { a: 1 });</code> and <code>Object.fromEntries([["a", 1]])</code>	new objects, shallow
9	<code>new Map(m)</code> / <code>new Set(s)</code>	<code>const m2 = new Map(m);</code>	<code>m2 !== m</code> ; <code>m2.set("x", 1)</code> leaves <code>m</code> alone
10	<code>structuredClone</code>	<code>const o = { a: { b: 1 } }; const c = structuredClone(o); c.a.b = 9;</code>	<code>o.a.b</code> is still 1 (deep)
11	JSON round trip	<code>JSON.parse(JSON.stringify({ d: new Date(0) }));</code>	deep, but <code>d</code> becomes the string <code>"1970-01-01T00:00:00.000Z"</code> ; <code>undefined</code> values and functions vanish
12	<code>new C()</code>	<code>const a = new User(); const b = new User();</code>	<code>a !== b</code>
13	function literal	<code>const f = () => {}; const g = () => {};</code>	<code>f !== g</code>
14	<code>split</code> / <code>groupBy</code>	<code>"a,b".split(",")</code> and <code>Object.groupBy(arr, f)</code>	new array / new object
15	function returning a literal	<code>function f() { return []; }</code>	<code>f() !== f()</code> : a new array per call

This line made an ALIAS (shared with something else)

#	Form	Concrete example	Result
16	<code>const b = a;</code>	<code>const a = [1]; const b = a; b.push(2);</code>	<code>a</code> is <code>[1, 2]</code>
17	function parameter	<code>function f(xs) { xs.push(1); } const a = []; f(a);</code>	<code>a</code> is <code>[1]</code> : the caller's array changed
18	<code>find</code> / <code>[i]</code> / <code>at</code>	<code>const u = users.find(p);</code> and <code>users[0]</code>	<code>u === users[k]</code> : the same user object; <code>u.name = "x"</code> changes the one in <code>users</code>
19	<code>map.get(k)</code>	<code>const m = new Map([["k", { n: 1 }]]); const v = m.get("k"); v.n = 2;</code>	<code>m.get("k").n</code> is 2
20	<code>o.k</code> where the value is an object	<code>const o = { inner: { x: 1 } }; const i = o.inner;</code>	<code>i === o.inner</code> ; <code>i.x = 2</code> changes <code>o.inner.x</code>
21	<code>const { inner } = o</code>	same <code>o</code>	<code>inner === o.inner</code> ; identical to row 20
22	returning internal state	<code>class A { items = []; get() { return this.items; } }</code> then <code>a.get().push(1)</code>	<code>a.items</code> is <code>[1]</code> : the caller edited internal state
23	cache hands out the stored object	<code>cache.set(k, obj); return obj;</code> then the caller does <code>obj.n = 5</code>	the cache entry now has <code>n = 5</code> for every later hit
24	closure captures an object	<code>const o = { n: 1 }; const f = () => o.n; o.n = 2; f()</code>	2: the closure reads the live object
25	<code>sort</code> / <code>reverse</code> / <code>fill</code> return the same array	<code>const a = [3, 1]; const s = a.sort();</code>	<code>s === a</code> is true ; both are <code>[1, 3]</code>
26	<code>Object.assign(target, src)</code>	<code>const t = {}; const r = Object.assign(t, { a: 1 });</code>	<code>r === t</code> is true ; <code>t</code> is <code>{ a: 1 }</code>
27	elements of a <code>filter</code> / <code>map(x => x)</code> result	<code>const f = users.filter(p); f[0].name = "z";</code>	the matching user in <code>users</code> has name <code>"z"</code>

#	Form	Concrete example	Result
28	<code>forEach</code> callback parameter	<code>orders.forEach(o => { o.status = "paid"; });</code>	every order in <code>orders</code> is now paid
29	module-level state	<code>const state = {}; export function add(k) { state[k] = 1; }</code>	every importer and every request shares <code>state</code>
30	<code>this</code> in a method	<code>class A { n = 0; inc() { this.n++; } }</code>	<code>a.inc()</code> changes that instance for everyone holding <code>a</code>

Mutations that reach every alias

#	Form	Concrete example	Result
31	property or index set	<code>const a = [1]; const b = a; b[0] = 9;</code>	<code>a[0]</code> is 9
32	<code>delete</code>	<code>const o = { k: 1 }; const p = o; delete o.k;</code>	<code>p.k</code> is <code>undefined</code>
33	push / pop / shift / unshift / splice	<code>const a = [1, 2, 3]; const b = a; a.splice(0, 1);</code>	<code>b</code> is <code>[2, 3]</code>
34	sort / reverse / fill / copyWithin	<code>const a = [3, 1]; const b = a; a.sort();</code>	<code>b</code> is <code>[1, 3]</code>
35	<code>length = n</code>	<code>const a = [1, 2]; const b = a; a.length = 0;</code>	<code>b</code> is <code>[]</code>
36	<code>Map</code> / <code>Set</code> mutators	<code>const m = new Map(); const n = m; m.set("a", 1);</code>	<code>n.get("a")</code> is 1
37	<code>Date</code> setters	<code>const d = new Date(0); const e = d; d.setFullYear(2000);</code>	<code>e.getFullYear()</code> is 2000
38	<code>Object.assign(o, src)</code>	<code>const o = { a: 1 }; const p = o; Object.assign(o, { a: 2 });</code>	<code>p.a</code> is 2
39	<code>freeze</code> / <code>defineProperty</code>	<code>const o = { a: 1 }; Object.freeze(o); o.a = 2;</code>	<code>o.a</code> is still 1 (silently ignored, or <code>TypeError</code> in strict mode); <code>freeze</code> is shallow
40	nested set	<code>const o = { n: { k: 1 } }; const s = o.n; o.n.k = 2;</code>	<code>s.k</code> is 2
41	mutate each element	<code>const a = [{ v: 1 }, { v: 2 }]; const b = a; a.forEach(x => { x.v = 0; });</code>	<code>b[1].v</code> is 0
42	regex with <code>g</code>	<code>const re = /a/g; re.exec("aa"); re.lastIndex</code>	1: calling <code>exec</code> advanced shared state; a module-level <code>/a/g</code> used in two places interferes with itself

Who the “else” usually is

#	Holder	Concrete example	How to spot it
43	the caller	<code>function f(opts) { opts.retries = 5; } then f(config)</code>	<code>config.retries</code> is now 5; the object came in as a parameter
44	the module (every request)	<code>const cache = new Map();</code> at the top of a file	declared outside any function
45	the class (every method)	<code>this.items.push(x)</code> in one method, read in another	<code>this.</code> on a field
46	the cache	<code>const v = cache.get(k); v.hits++;</code>	the object came from a get on a long-lived <code>Map</code>
47	the original array	<code>const first = users.find(p); first.active = false;</code>	the object was an element from <code>find</code> , <code>[i]</code> , or <code>filter</code>
48	a previous caller	<code>const DEFAULT = {}; function f() { return DEFAULT; }</code>	<code>f()</code> returns the SAME object every call
49	a listener or timer	<code>setInterval(() => use(big), 1000);</code>	the closure keeps <code>big</code> alive and shared
50	defaults / config	<code>const opts = defaults; opts.x = 1;</code>	missing <code>{ ...defaults }</code> ; <code>defaults.x</code> is now 1 for everyone

Q7. Was the result kept?

Kept (the value goes somewhere)

#	Form	Concrete example	Where it went
1	assigned	<code>const x = f(); o.k = f();</code>	into <code>x</code> ; into <code>o.k</code>
2	returned	<code>return f();</code>	to the caller
3	awaited into a label	<code>const x = await f();</code>	the resolved value into <code>x</code>
4	passed as an argument	<code>g(f()); arr.push(f()); res.json(f());</code>	into <code>g</code> ; onto the array; into the response
5	used in a condition	<code>if (f()) {}; while (f()) {}; f() ? a : b</code>	decided the branch
6	used in an expression	<code>a + f(); `\${f()}`</code>	became part of a larger value
7	stored	<code>map.set(k, f()); arr[i] = f();</code>	into the Map; into the slot
8	spread	<code>[...f()]; ({ ...f() })</code>	copied into a new container
9	destructured	<code>const { a } = f();</code>	<code>a</code> holds <code>f().a</code>
10	chained	<code>f().g()</code>	the next call consumed it
11	yielded	<code>yield f();</code>	handed to the generator's caller
12	thrown	<code>throw f();</code>	became the exception

Not kept, and that is fine (the effect was the point)

#	Form	Concrete example	What the effect was
13	mutating call	<code>arr.push(x);</code> with <code>arr = [1]</code> , <code>x = 2</code>	<code>arr</code> became <code>[1, 2]</code> ; the returned length 2 is dropped
14	property set	<code>o.k = v;</code>	<code>o.k</code> now holds <code>v</code>
15	side-effect call	<code>console.log(x); res.json(d); emitter.emit("e"); clearTimeout(id);</code>	printed; sent; notified listeners; cancelled the timer
16	awaited, value dropped	<code>await save(x);</code>	waited until <code>save</code> finished
17	<code>void</code>	<code>void f();</code>	explicit "I am ignoring this"
18	sync call with documented effect	<code>logger.info("started");</code>	wrote a log line

Not kept, and that is the bug

#	Form	Concrete example	What went wrong
19	non-mutating array method	<code>[1, 2].map(x => x * 2);</code> (also <code>filter</code> , <code>slice</code> , <code>concat</code> , <code>flat</code>)	a new array <code>[2, 4]</code> built and dropped; dead line
20	non-mutating twins	<code>arr.toSorted();</code> and <code>arr.toReversed();</code>	new array dropped
21	string method	<code>let s = " a "; s.trim();</code>	<code>s</code> is still <code>" a "</code> ; the <code>"a"</code> was dropped
22	copy dropped	<code>({ ...o });</code> and <code>[...a];</code> and <code>Object.assign({}, o);</code>	a copy built and dropped
23	clone / parse dropped	<code>structuredClone(o);</code> and <code>JSON.parse(s);</code>	result dropped (and <code>JSON.parse</code> may still throw)
24	lookup dropped	<code>arr.find(p);</code> and <code>arr.includes(x);</code> and <code>map.get(k);</code>	looked something up and ignored the answer
25	async call, no await	<code>save(x);</code>	Promise dropped; rejection unhandled; next line runs too early
26	<code>fetch</code> dropped	<code>fetch(url);</code>	request sent; response and errors ignored
27	async <code>map</code> dropped	<code>ids.map(async id => save(id));</code>	an array of Promises dropped; nothing waits
28	<code>.then</code> with no <code>.catch</code>	<code>p.then(g);</code>	a rejection of <code>p</code> or <code>g</code> is unhandled
29	pure expression	<code>x;</code> and <code>a + b;</code> and <code>o.k;</code>	evaluated and discarded
30	<code>new</code> dropped	<code>new Date();</code> and <code>new User();</code>	an object built and dropped
31	async IIFE, no catch	<code>(async () => { await save(x); })();</code>	Promise dropped; errors unhandled
32	kept, but misleading: <code>sort</code>	<code>const sorted = arr.sort();</code>	kept, but <code>sorted === arr</code> : both reordered; not a copy

#	Form	Concrete example	What went wrong
33	kept, but not what you think: <code>push</code>	<code>const n = arr.push(x);</code>	<code>n</code> is the new LENGTH (a number), not the array

The note, with every field and an example of each

After Q7 write:

label holds: value · own or shared · can be nothing · ticket or value.

Field	Options	Example
value	a concrete example	<code>6, "ABC", { id: "42", name: "Ann" }, [1, 10, 9], Promise<User>, undefined</code>
own or shared	own (new address) / shared with the caller, <code>users[]</code> , the cache, module state, <code>defaults</code>	<code>user holds: an element of users[], shared</code>
can be nothing	yes (and why) / no	<code>yes: find miss</code>
ticket or value	value / ticket not awaited / ticket awaited on line N	<code>ticket, not awaited</code>

Whole notes, written out: `user holds: a User object, shared with users[], can be undefined (find miss), value and p holds: Promise<User>, own, cannot be nothing, ticket not awaited (BUG).`

The next line's Q4 reads "ticket or value", its Q5 reads "can be nothing", and its Q6 reads "own or shared". That is the entire hand-off.